

PROJECT REPORT ON

ResolveNow: Your Platform for Online Complaints

SUBMITTED BY

Team ID: LTVIP2026TMIDS53382

Team Size: 4

Team Members : G. Vasantha Lakshmi

P.Sashikala

K.Pradeep

D.Uday Kiran

INTRODUCTION

The Online Complaint Registration and Management System, **ResolveNow**, is a user-friendly software solution designed to streamline the process of submitting, tracking, and resolving complaints or issues encountered by individuals or organizations. It provides a centralized platform for efficient complaint management, allowing users to securely register complaints, track their progress in real-time, and interact with assigned agents for issue resolution.

This system helps optimize the complaint handling process and empowers organizations to develop a safety management system to efficiently resolve customer complaints, while staying in line with industry guidelines and regulatory compliance obligations. With features such as automatic notifications, intelligent complaint routing, and robust security measures, this system ensures timely and effective handling of complaints while prioritizing user data security and confidentiality.

Key Features

- **User Registration:** Users can create secure accounts to submit complaints and track their progress from start to finish.
- **Complaint Submission:** Users can enter detailed descriptions of their complaints, including relevant information such as name, description of the issue, address, and attach supporting documents or images.
- **Tracking and Notifications:** Users can track the progress of their complaints in real-time, view updates, and receive notifications via email or SMS whenever there are status changes or resolutions.
- **Agent Interaction:** Users can directly interact with the agent assigned to their complaint through a built-in messaging system.
- **Assigning and Routing:** The system assigns complaints to the appropriate department or personnel. It can utilize intelligent routing to ensure efficient allocation of resources.
- **Security and Confidentiality:** The system ensures the security and confidentiality of all user data and complaint information through robust measures such as user authentication, data encryption, role-based access controls, and compliance with data protection regulations.

SCENARIO-BASED CASE STUDY

Scenario: John, a customer, recently encountered a problem with a product he purchased online. He notices a defect in the item and decides to file a complaint using the ResolveNow system.

1. User Registration and Login:

- John visits the ResolveNow website and clicks on the "Sign Up" button.
- He fills out the registration form, providing his full name, email address, and a secure password.
- After submission, John receives a verification email to confirm his account.
- He then logs into the system using his credentials.

2. Complaint Submission:

- Upon logging in, John is directed to his dashboard where he selects the option to register a new complaint.
- He fills out the complaint form, describing the defect in detail, attaching images of the faulty product, and providing his contact details and the purchase date.
- After reviewing the information, John submits the complaint.

3. Tracking and Notifications:

- John immediately receives an on-screen confirmation and an email notification that his complaint has been successfully registered with a unique tracking ID.
- He navigates to the "My Complaints" section, where he can see the status of his complaint is "Submitted."
- John receives further email notifications when the status changes, for instance, when it is assigned to an agent.

4. Interaction with Agent:

- A customer service agent, Sarah, is assigned to handle John's complaint. Her dashboard is updated with the new case.
- Sarah reviews the details and contacts John through the system's built-in messaging feature to ask for more information.
- John is notified of Sarah's message, logs in, and communicates with her directly, discussing the issue further.

5. Resolution and Feedback:

- After investigating, the company approves a replacement for the defective product.
- Sarah updates the complaint status to "Resolved" and informs John of the outcome and the steps for receiving his replacement.
- John receives a final notification and is prompted to provide feedback on his experience, expressing satisfaction with the prompt and helpful service.

6.Admin Management:

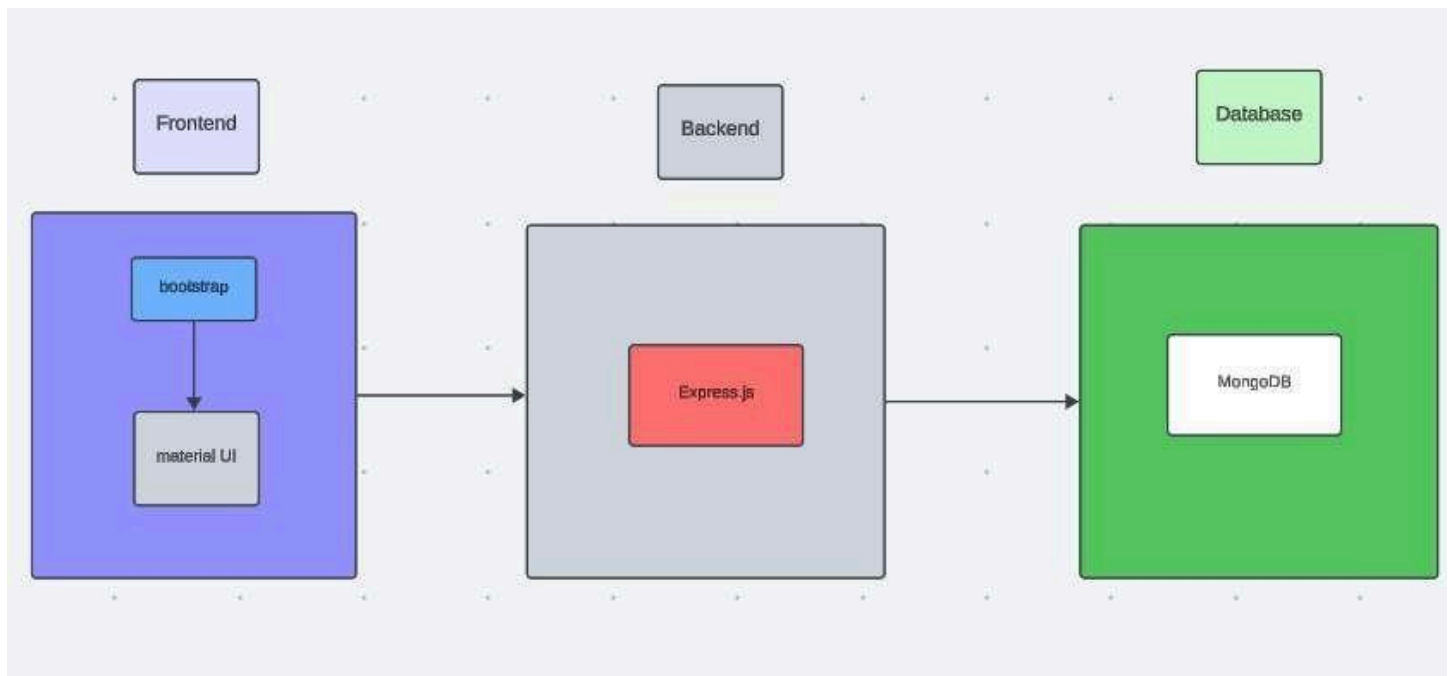
- Meanwhile, a system administrator monitors all platform activities from an admin dashboard.
- The admin oversees the assignment of complaints to agents like Sarah, ensuring balanced workloads and timely responses.
- They also monitor overall system performance, ensuring compliance with platform policies and generating reports on complaint trends.

TECHNICAL ARCHITECTURE

The technical architecture of our online complaint registration and management app follows a client-server model, where the frontend serves as the client and the backend acts as the server.

The frontend encompasses the user interface and presentation, incorporating the **Axios** library to connect with the backend seamlessly using RESTful APIs. It utilizes **Bootstrap** and **Material UI** to establish a responsive, real-time, and superior UI experience for any user, whether they are a customer, agent, or admin.

On the backend, we employ the **Express.js** framework on top of **Node.js** to handle the server-side logic, API routing, and communication. For data storage and retrieval, our backend relies on **MongoDB**, which allows for efficient and scalable storage of user data, complaint details, and message histories.

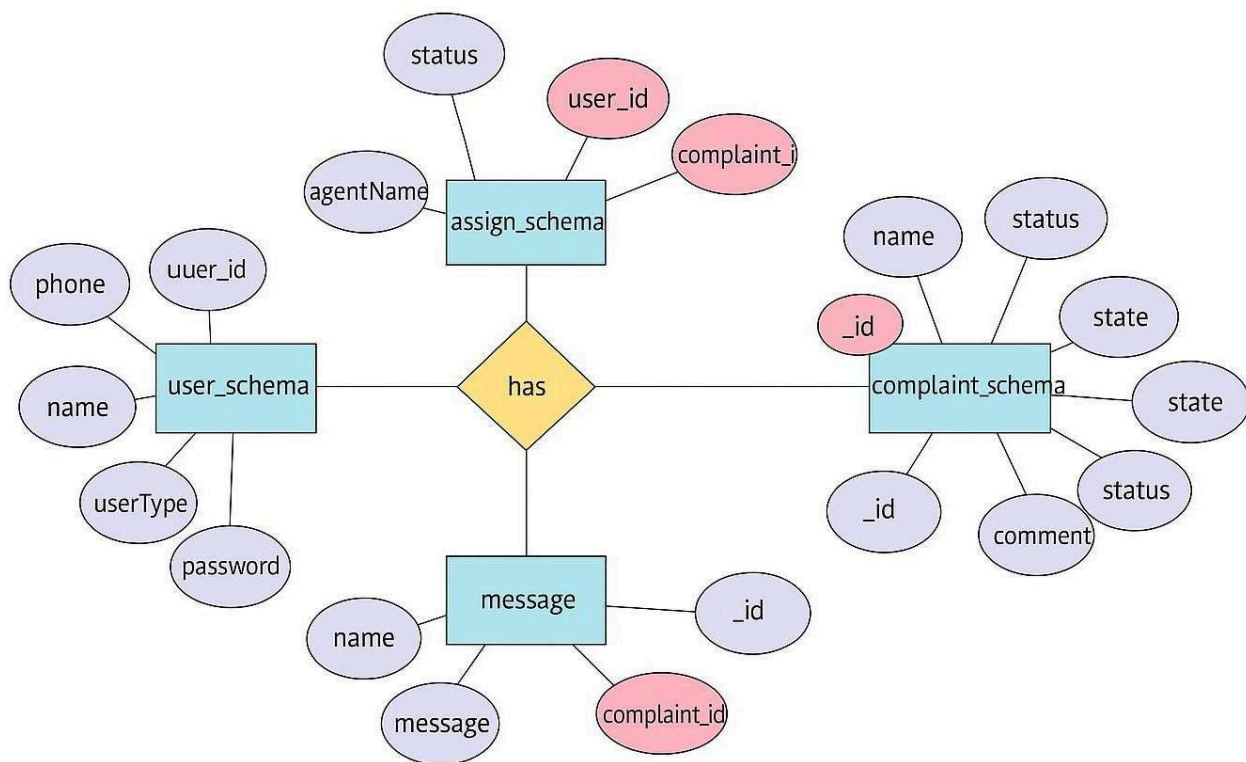


ER Diagram

The Entity-Relationship (ER) diagram for the ResolveNow app outlines three primary entities: **Users**, **Complaints**, and **Agents**, along with their attributes and relationships.

- **Users:** Stores basic user information (userId, name, email, password, role). The role attribute differentiates between 'customer', 'agent', and 'admin'.

- **Complaints:** Holds all details about a complaint, including complaintId, userId (foreign key to Users), agentId (foreign key to Agents), description, status, dateSubmitted, and dateResolved.
- **Messages:** Contains the communication history for a complaint, with fields like messageId, complaintId, senderId, receiverId, and timestamp.



PRE REQUISITES :

NODE.JS AND NPM:

- Node.js is a JavaScript runtime that allows you to run JavaScript code on the server-side. It provides a scalable platform for network applications.
- npm (Node Package Manager) is required to install libraries and manage dependencies.
- Download Node.js: [Node.js Download](#)
- Installation instructions: [Installation Guide](#)
- Run npm init to set up the project and create a package.json file.

EXPRESS.JS:

- Express.js is a web application framework for Node.js that helps you build APIs and web applications with features like routing and middleware.

- Install Express.js to manage backend routing and API endpoints.
- Install Express:
- Run npm install express

MONGODB:

- MongoDB is a NoSQL database that stores data in a JSON-like format, making it suitable for storing data like user profiles, doctor details, and appointments.
- Set up a MongoDB database for your application to store data.
- Download MongoDB: [MongoDB Download](#)
- Installation instructions: [MongoDB Installation Guide](#)

MOMENT.JS:

.

- Moment.js is a JavaScript package for handling date and time operations, allowing easy manipulation and formatting.
- Install Moment.js for managing date-related tasks, such as appointment scheduling.
- Moment.js Website: [Moment.js Documentation](#)

REACT.JS:

- React.js is a popular JavaScript library for building interactive and reusable user interfaces. It enables the development of dynamic web applications.
- Install React.js to build the frontend for your application. ● [React.js Documentation: Create a New React App](#)

ANTD (ANT DESIGN):

- Ant Design is a UI library for React.js, providing a set of reusable components to create user-friendly and visually appealing interfaces.
- Install Ant Design for UI components such as forms, tables, and modals. ● [Ant Design Documentation: Ant Design React](#)

HTML, CSS, AND JAVASCRIPT:

- Basic knowledge of HTML, CSS, and JavaScript is essential to structure, style, and add interactivity to the user interface.

DATABASE CONNECTIVITY (MONGOOSE):

- Use Mongoose, an Object-Document Mapping (ODM) library, to connect your Node.js backend to MongoDB for managing CRUD operations.
- [Learn Database Connectivity: Node.js + Mongoose + MongoDB](#)

FRONT-END FRAMEWORKS AND LIBRARIES:

- React.js will handle the client-side interface for managing doctor bookings, viewing appointment statuses, and providing an admin dashboard.
- You may use Material UI and Bootstrap to enhance the look and feel of the application.

SETUP AND INSTALLATION INSTRUCTIONS :

CLONE THE PROJECT REPOSITORY:

- Download the project files from GitHub or clone the repository using Git.

INSTALL DEPENDENCIES:

- Navigate to the frontend and backend directories and install all required dependencies for both parts of the application.
- Frontend:
- Navigate to the frontend directory and run npm install.
- Backend:
- Navigate to the backend directory and run npm install.

START THE DEVELOPMENT SERVER:

- After installing the dependencies, start the development server for both frontend and backend.
- Frontend will run on <http://localhost:3000>.
- Backend will run on <http://localhost:8001> or the specified port.

ACCESS THE APPLICATION:

- After running the servers, access the Doctor Appointment Webpage in your browser at <http://localhost:3000> for the frontend interface and <http://localhost:8001> for backend API services.

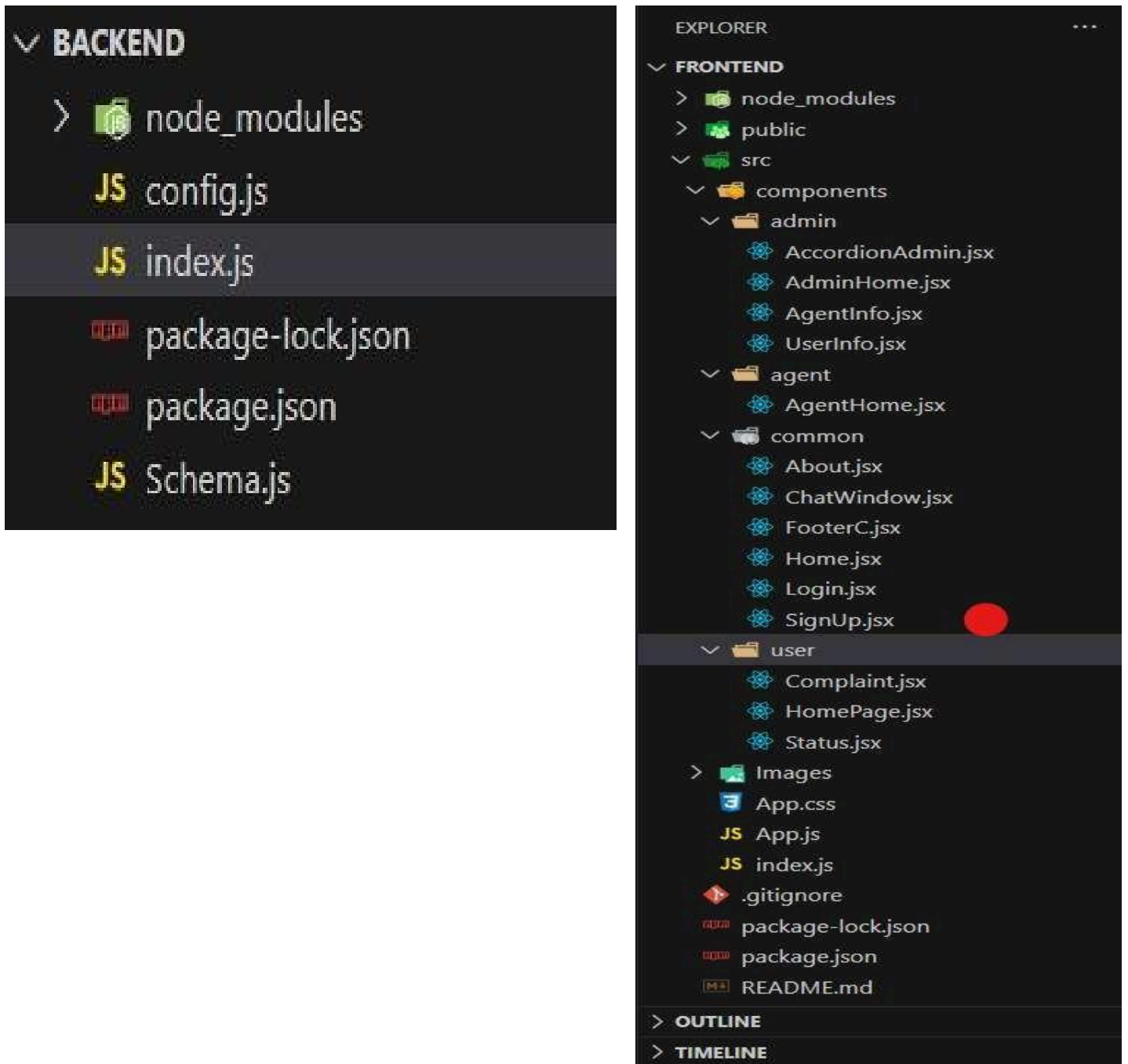
PROJECT IMPLEMENTATION

Milestones

1. **Project Setup & Configuration:** Establishing the development environment, initializing frontend and backend repositories, and installing all necessary libraries and tools (Node.js, React, MongoDB).
2. **Backend Development:** Setting up the Express.js server, defining API routes, configuring middleware, and implementing user authentication with JWT.
3. **Database Development:** Designing and implementing MongoDB schemas for Users, Complaints, and Messages using Mongoose. Building the CRUD (Create, Read, Update, Delete) functionalities for each model.
4. **Frontend Development:** Building the React application structure, creating reusable UI components for forms, dashboards, and modals using Material UI. Implementing state management and integrating with the backend APIs using Axios.

5. **Integration & Testing:** Connecting all components, performing end-to-end functional testing of all user journeys, fixing bugs, and ensuring the application is stable and performs well.

PROJECT STRUCTURE :



APPLICATION FLOW:

Online Complaint Registration and Management System

1. Customer/Ordinary User:

- a. **Role:** Create and manage complaints, interact with agents, and manage profile information.
- b. **Flow:**
 - i. **Registration and Login:**
 - 1. Create an account by providing necessary information such as email and password.
 - 2. Log in using the registered credentials.
 - ii. **Complaint Submission:**
 - 1. Fill out the complaint form with details of the issue, including description, contact information, and relevant attachments.
 - 2. Submit the complaint for processing.
 - iii. **Status Tracking:**
 - 1. View the status of submitted complaints in the dashboard or status section.
 - 2. Receive real-time updates on the progress of complaints.
 - iv. **Interaction with Agents:**
 - 1. Connect with assigned agents directly using the built-in messaging feature.
 - 2. Discuss complaints further and provide additional information or clarification.
 - v. **Profile Management:**
 - 1. Manage personal profile information, including details and addresses.

2. Agent:

- a. **Role:** Manage complaints assigned by the admin, communicate with customers, and update complaint statuses.
- b. **Flow:**
 - i. **Registration and Login:**
 - 1. Create an account using email and password.
 - 2. Log in using the registered credentials.
 - ii. **Complaint Management:**
 - 1. Access the dashboard to view and manage complaints assigned by the admin.
 - 2. Communicate with customers regarding their complaints through the chat window.
 - iii. **Status Update:**
 - 1. Change the status of complaints based on resolution or progress.
 - 2. Provide updates to customers regarding the status of their complaints.
 - iv. **Customer Interaction:**
 - 1. Respond to inquiries, resolve issues, and address feedback from customers.

3. Admin:

- a. **Role:** Oversee the overall operation of the complaint registration platform, manage complaints, users, and agents, and enforce platform policies.
- b. **Flow:**
 - i. **Management and Monitoring:**
 - 1. Monitor and moderate all complaints submitted by users.
 - 2. Assign complaints to agents based on workload and expertise.
 - ii. **Complaint Assignment:**

1. Assign complaints to the desired agents for resolution.
2. Ensure timely and efficient handling of complaints.

iii. User and Agent Management:

1. Manage user and agent accounts, including registration, login, and profile information.
2. Enforce platform policies, terms of service, and privacy regulations.

iv. Continuous Improvement:

1. Implement measures to improve the platform's functionality, user experience, and security measures.
2. Address any issues or concerns raised by users or agents for better service delivery.

SETUP & CONFIGURATION :

Setting up the ResolveNow Online Complaints Webpage involves configuring both the Frontend (React.js) and Backend (Node.js, Express.js, MongoDB) to ensure the application runs smoothly. Below are the steps to set up and configure the environment for your project.

FRONTEND CONFIGURATION :

INSTALLATION :

Clone the Repository:

- Clone the project from GitHub to your local machine:
- bash
- Copy code
- `git clone <your-repository-url>`
- Replace `<your-repository-url>` with the URL of your project repository.

Navigate to Frontend Directory:

- After cloning, navigate to the frontend folder where the React.js app is located:
- bash
- Copy code
- `cd book-a-doctor/frontend`

Install Dependencies:

- Use npm (Node Package Manager) to install the necessary dependencies:
- bash
- Copy code
- `npm install`
- This will install all the required libraries, such as React, React Router, Ant Design, and others.

Run the React Development Server:

- To start the frontend server and run the React application:
- bash
- Copy code

npm start

- The application will be available at <http://localhost:3000> in your browser.

BACKEND CONFIGURATION :

INSTALLATION :

Navigate to Backend Directory:

- Move to the backend folder of your project:
- bash
- Copy code
- cd book-a-doctor/backend • Install Dependencies:
- Install the necessary backend dependencies using npm:
- bash
- Copy code
- npm install
- This will install all required libraries for Node.js and Express.js, such as express, mongoose, bcryptjs, jsonwebtoken, etc.

Configure MongoDB :

- Ensure you have MongoDB installed on your local machine or use a cloud-based MongoDB service like MongoDB Atlas.
- In the backend, open the configuration file (e.g., config/db.js or .env) and configure the connection URL for MongoDB:
- bash
- Copy code
- MONGO_URI=mongodb://localhost:27017/doctor_appointment
- If you are using MongoDB Atlas, replace the localhost part with your MongoDB Atlas connection string.

Set Up Environment Variables:

- Create a .env file in the backend directory to store environment-specific variables such as:
- bash
- Copy code
- PORT=5000
- JWT_SECRET=your_jwt_secret
- Make sure to replace your_jwt_secret with a strong secret key for JWT authentication.

Run the Backend Server:

- Start the backend server by running:
- bash
- Copy code
- npm start

- The backend server will be running at <http://localhost:5000>.

DATABASE CONFIGURATION (MONGODB) :

Install MongoDB (Local Installation):

If you are using a local MongoDB instance, download and install it from the official MongoDB website:
Download MongoDB

Set Up MongoDB Database:

- After installation, start the MongoDB service:
- bash
- Copy code
- mongod
- This will run MongoDB on the default port 27017.

MongoDB Atlas (Cloud-based MongoDB):

- If you prefer to use MongoDB Atlas, create an account on MongoDB Atlas and create a cluster.
- Once the cluster is set up, get the connection string and replace it in the backend's .env file:
- bash
- Copy code
- MONGO_URI=<your-mongodb-atlas-connection-string>

FINAL CONFIGURATION & RUNNING THE APP

Run Both Servers:

- The React frontend and Node.js backend servers should run simultaneously for the app to function correctly.
- You can open two terminal windows or use tools concurrently to run both servers together.
- To install concurrently, run:
- bash
- Copy code
- npm install concurrently --save-dev
-

In your package.json file, add a script to run both servers:

```
json
Copy code
"scripts": {
  "start": "concurrently \"npm run server\" \"npm run client\"",
  "server": "node backend/server.js",
  "client": "npm start --prefix frontend"
}
```

Start Both Servers:

- Now you can run the following command in the root directory to start both the backend and frontend:
- bash
- Copy code • npm start
- The backend will be available at <http://localhost:5000>, and the frontend at <http://localhost:3000>.

VERIFYING THE APP :

Check Frontend:

- Open your browser and go to <http://localhost:3000>. The ResolveNow application should load with the list of **complaints**, a form to **submit a new complaint**, and **status updates** for existing issues.

Check Backend:

- Open Postman or any API testing tool to verify if the backend endpoints are working correctly, such as user login, doctor registration, and appointment creation.

ADDITIONAL SETUP :

- Version Control:
- If you haven't already done so, initialise a Git repository in the root of your project:
- bash
- Copy code
- git init

Add your project files and commit them:

- bash • Copy code • git add .
- git commit -m "Initial commit" • Deployment (Optional):
- If you want to deploy your app, consider using cloud platforms like Heroku, AWS, or Vercel for frontend hosting and backend API deployment.

FOLDER SETUP :

The folder structure for your Doctor Appointment Webpage project will include separate folders for the frontend and backend components to keep the code organised and modular. Here's how to set it up:

PROJECT ROOT STRUCTURE :

Create the Main Folders:

- In your project's root directory, create two main folders: frontend and backend.
- plaintext
- Copy code
- project-root/
 - ├── frontend/
 - └── backend/

BACKEND SETUP :

- Install Necessary Packages in the Backend Folder:
- Navigate to the backend folder and install the following essential packages:
- plaintext
- Copy code
- backend/
 - |— config/
 - |— controllers/
 - |— models/
 - |— routes/
 - |— middleware/
 - |— uploads/
 - |— server.js
 - |— .env

Packages to Install :

- cors: To enable cross-origin requests.
 - bcryptjs: For securely hashing user passwords.
 - express: A lightweight framework to handle server-side routing and API management.
 - dotenv: For loading environment variables.
 - mongoose: To connect and interact with MongoDB.
 - multer: To handle file uploads.
 - nodemon: A utility to auto-restart the server upon code changes (for development).
 - jsonwebtoken: To manage secure, stateless user authentication.
- Installation Commands

Run these commands in the backend folder:

.

- bash
- Copy code
- npm init -y
- npm install cors bcryptjs express dotenv mongoose multer jsonwebtoken ● npm install --save-dev nodemon

FRONTEND SETUP :

- React Project Initialization:
- Navigate to the frontend folder and initialise a new React application:
- plaintext
- Copy code
- frontend/
 - |— public/
 - |— src/
 - |— components/
 - |— pages/

```

|
|   └── services/
|   └── App.js
|   └── .env
|   └── package.json

```

- Setting Up the Frontend Project
- In the frontend folder, run the following commands to set up and install any initial dependencies:
 - bash

PROJECT FLOW :

Project Demo :

- Before diving into development, you can view a demo of the project to understand its functionality and user interactions.
- Project FlowDemo Video :
https://drive.google.com/file/d/1WF_4ivsguyTKDmhaO-TNyLQPC9JA1lGs/view?usp=sharing
- Project Code Repository :
[ResolveNow Online Complaint Care Using MERN STACK](#)
- The source code for this project can be accessed and cloned from GitHub, providing a base structure and example code for further customization and understanding.
 - GitHub Repository: Source Code

Video Tutorials :

- For a step-by-step guide on setting up and working with this project, follow the video tutorials below:
- Video Guide 1: Setting Up the Backend
- Video Guide 2: Configuring Frontend and API Endpoints
- Video Guide 3: Testing and Deployment
- These resources should give you a solid foundation and clear understanding of the project structure and workflow before development begins.

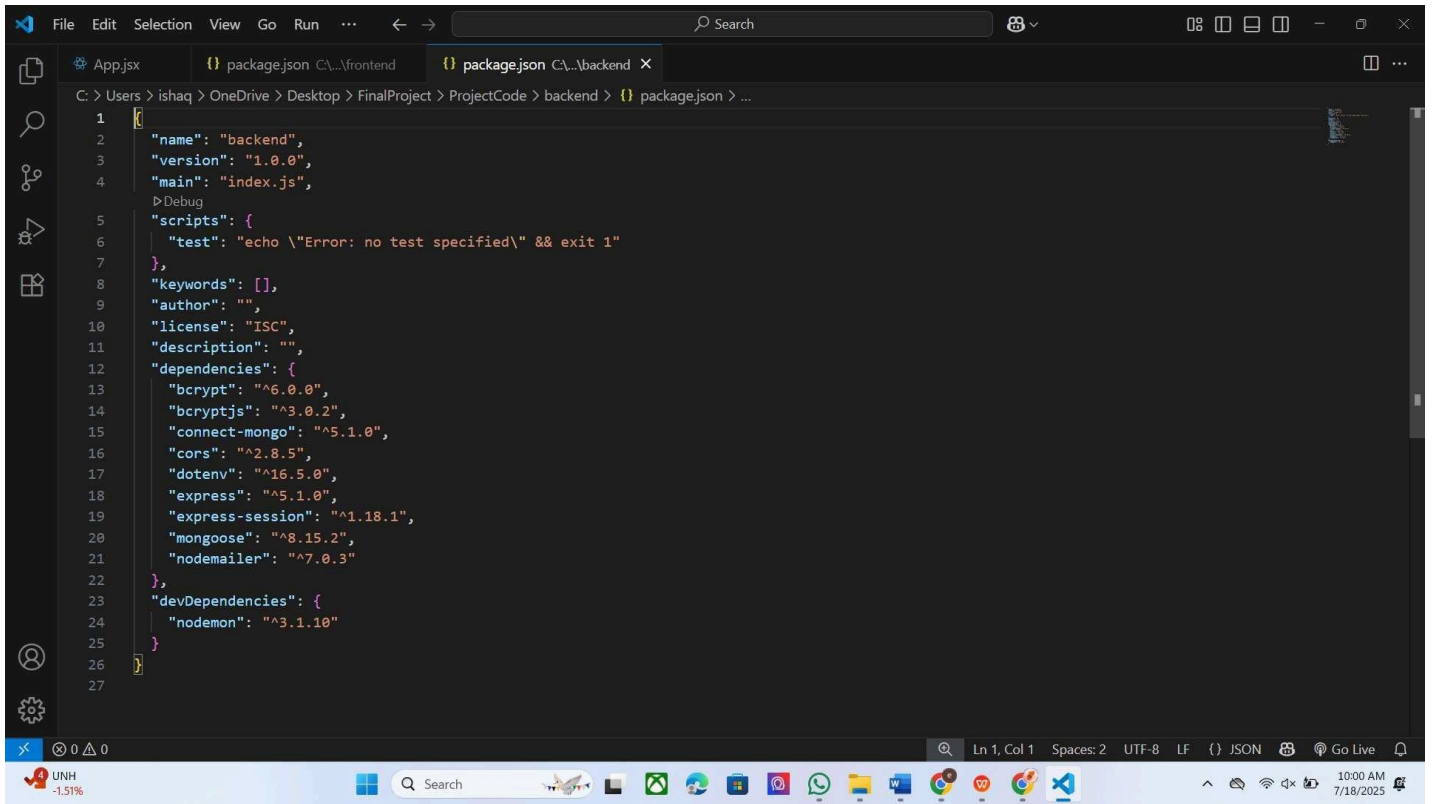
MILESTONE 1: PROJECT SETUP AND CONFIGURATION :

- Setting up a structured environment is crucial for any application. By organising the project into separate folders for the frontend and backend, we ensure that the codebase is manageable and scalable.

```

1  {
2    "name": "frontend",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "proxy": "http://localhost:5000",
7    "scripts": {
8      "dev": "vite",
9      "build": "vite build",
10     "lint": "eslint .",
11     "preview": "vite preview"
12   },
13   "dependencies": {
14     "@fortawesome/fontawesome-free": "^6.7.2",
15     "@tailwindcss/vite": "^4.1.10",
16     "axios": "^1.10.0",
17     "lucide-react": "^0.518.0",
18     "react": "^19.1.0",
19     "react-dom": "^19.1.0",
20     "react-router-dom": "^7.6.2"
21   },
22   "devDependencies": {
23     "@eslint/js": "^9.25.0",
24     "@tailwindcss/postcss": "^4.1.10",
25     "@types/react": "^19.1.8",
26     "@types/react-dom": "^19.1.2",
27     "@types/react-router-dom": "^5.3.3",
28     "@vitejs/plugin-react": "^4.4.1",
29     "autoprefixer": "^10.4.21",

```

A screenshot of a code editor window showing a package.json file. The file is for a project named 'backend' with version '1.0.0'. It includes a 'main' field pointing to 'index.js', a 'scripts' section with a 'test' script, and lists various dependencies like bcrypt, connect-mongo, cors, dotenv, express, express-session, mongoose, and nodemailer. It also lists devDependencies like nodemon. The editor has a dark theme and a sidebar on the left with icons for Explorer, Search, Source Control, and Run and Debug. The bottom status bar shows 'Ln 1, Col 1', 'Spaces: 2', 'UTF-8', 'LF', and 'JSON'.

LIBRARY AND TOOL INSTALLATION:

Backend Libraries:

- Node.js: Provides a runtime environment to run JavaScript code on the server side.
- MongoDB: A NoSQL database, perfect for flexible and schema-less data storage, ideal for applications needing frequent updates and various data types.
- Bcrypt: Encrypts passwords for secure authentication, helping protect user data from potential breaches.
- Body-parser: Parses incoming request bodies, making it easy to access data in various formats like JSON.
- Frontend Libraries:
- React.js: Manages component-based UI creation, providing the flexibility to build reusable UI components.
- Material UI & Bootstrap: Provides styling frameworks, ensuring a consistent, responsive, and visually appealing design.
- Axios: Facilitates easy HTTP requests, allowing the frontend to communicate with the backend effectively.

MILESTONE 2: BACKEND DEVELOPMENT

The backend forms the core logic and data management for the **ResolveNow** application. A well-structured backend ensures efficient data handling, security, and scalability.

EXPRESS.JS SERVER SETUP:

- **Express Server:** Acts as a hub for all requests and responses, routing them to appropriate endpoints. It's essential for managing incoming requests from the frontend, processing them, and sending responses back.

- **Middleware Configuration:** Middleware like `body-parser` parses JSON data in requests, while `cors` enables cross-origin communication between the frontend and backend. This makes it easy to add additional functionality, like error handling or authentication checks, without interfering with core application logic.

API ROUTE DEFINITION:

- **Route Organization:** Organizing routes by functionality (e.g., `auth`, `users`, `complaints`, `messages`) keeps the codebase readable and easy to maintain. For instance, all authentication-related routes can reside in an `auth.js` file, ensuring each file has a single purpose.
- **Express Route Handlers:** Route handlers manage the flow of data between the client and server for fetching, creating, updating, and deleting records related to users, complaints, and messages.

DATA MODELS (SCHEMAS) WITH MONGOOSE:

- **User Schema:** Defines the structure for user data and includes fields for personal information and role-based access control (`customer`, `agent`, `admin`). Using a schema ensures consistent data storage and permissions.
- **Complaint and Message Models:** These models manage complex data interactions, including relationships between the user who filed the complaint, the agent assigned to it, and the conversation history. This allows for efficient querying and status tracking.
- **CRUD Operations:** CRUD functionalities (Create, Read, Update, Delete) provide a standard interface for handling all data operations, simplifying the manipulation of complaints and user profiles in a structured, predictable way.

USER AUTHENTICATION:

- **JWT Authentication:** JSON Web Tokens (JWT) will securely handle session management, ensuring that only verified users can access protected routes like viewing their complaint history or, for agents, managing assigned cases. JWTs are embedded in request headers, verifying each request without storing session data on the server.

ADMIN AND COMPLAINT MANAGEMENT:

- **Admin Privileges:** Administrators oversee user management, can view all complaints in the system, and are responsible for assigning agents to new complaints. Admin-specific routes ensure that these high-level actions are isolated and secure.
- **Complaint Lifecycle Management:** This functionality allows customers to track their submitted complaints, view status updates in real-time, and communicate directly with their assigned agents through the messaging system.

ERROR HANDLING:

- **Middleware for Error Handling:** Centralized error-handling middleware catches issues and sends meaningful error responses with appropriate HTTP status codes (like 404 for Not Found or 500 for Server Error), enabling better debugging and user feedback.

MILESTONE 3: DATABASE DEVELOPMENT

Using MongoDB as the database provides a flexible, schema-less structure, perfect for handling the varied data structures of users, complaints, and messages in the **ResolveNow** app.

SCHEMAS FOR DATABASE COLLECTIONS:

- **User Schema:** Defines fields for user information like `name`, `email`, `password`, and `role`. This schema allows fine-grained control over user data and permissions.
- **Complaint Schema:** This schema manages all complaint data, with fields linking the complaint to the submitting user and the assigned agent. It includes details like `description`, `status`, `dateSubmitted`, and `dateResolved` for efficient tracking.
- **Message Schema:** This schema organizes messages between users and agents, storing them by `complaintId` for a streamlined and threaded user-agent communication flow.

DATABASE COLLECTIONS IN MONGODB:

- MongoDB collections, such as `users`, `complaints`, and `messages`, provide a structured, NoSQL approach to data management, making it easy to scale as the volume of user interactions and submitted complaints grows.

MILESTONE 4: FRONTEND DEVELOPMENT

Frontend development focuses on creating an interactive, intuitive user experience for all user roles using a React-based user interface.

REACT APPLICATION SETUP:

- **Folder Structure and Libraries:** Setting up the initial React app structure (e.g., using `create-react-app`) and libraries ensures a smooth development workflow. By organizing files into `components`, `services`, and `pages`, the project becomes easy to navigate and maintain.
- **UI Component Libraries:** Material UI or Bootstrap will be used to offer pre-built components, enabling rapid UI development and a consistent, professional design across all screens.

UI COMPONENTS FOR REUSABILITY:

- **Reusable Components:** Each UI element, like the complaint submission form, the user dashboard, the chat window, and status badges, is designed as a reusable component. This modularity allows for efficient reuse across the app, reducing development time and ensuring consistency.
- **Styling and Layout:** Styling and layout components maintain a cohesive look and feel, contributing to the user experience with clean, intuitive visuals.

FRONTEND LOGIC IMPLEMENTATION:

- **API Integration:** `Axios` is used to make API calls to the backend, connecting UI components with data from the server to fetch complaints, send messages, and update user information.
- **Data Binding and State Management:** React's state management (e.g., using Context API or Redux) binds data to the UI, automatically updating what the user sees as they interact with the app.

MILESTONE 5: PROJECT IMPLEMENTATION AND TESTING

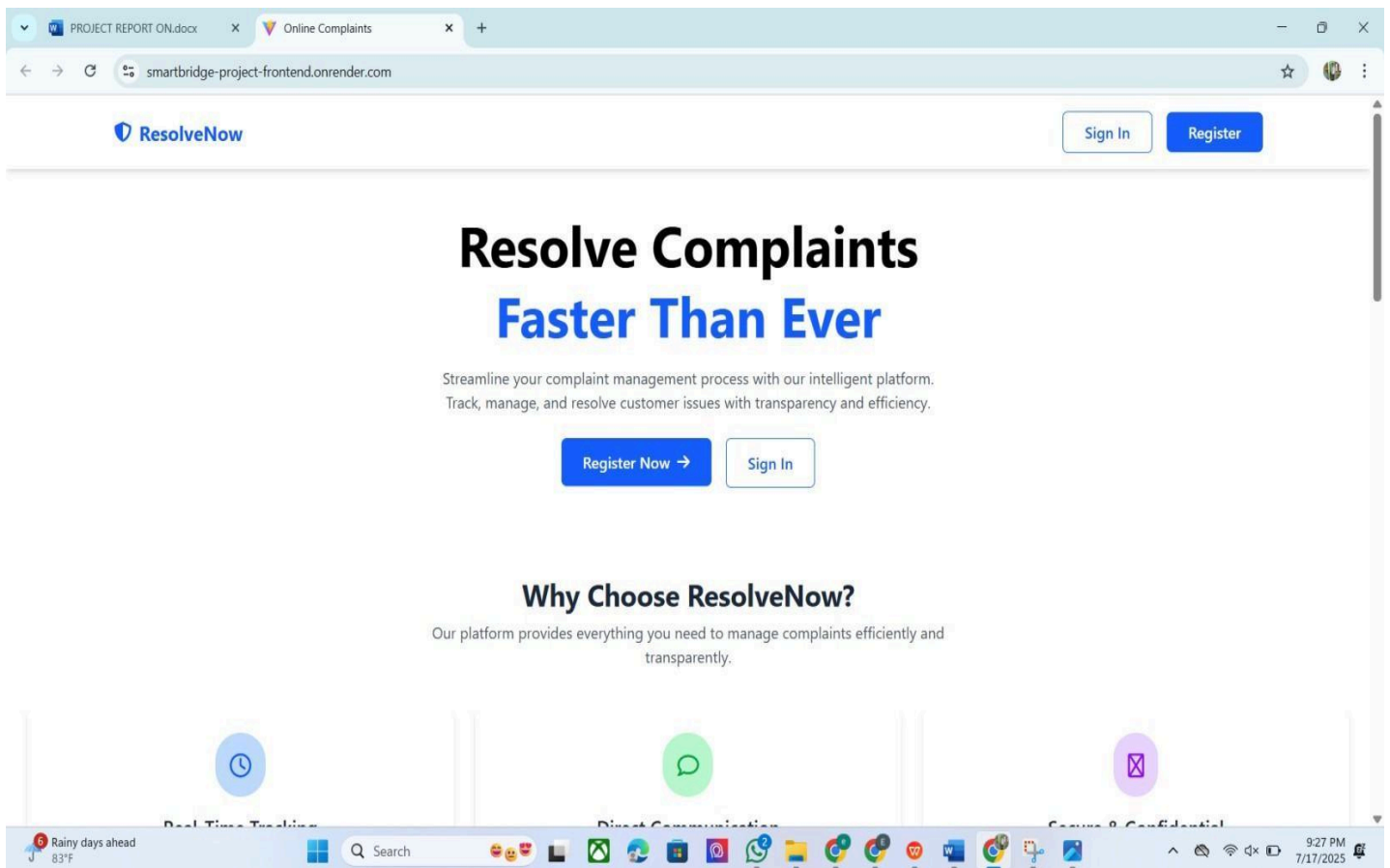
After completing development, running a final set of tests is crucial for identifying any bugs or issues before deployment.

VERIFY FUNCTIONALITY:

- Running the entire application ensures that each part (frontend, backend, database) works cohesively.
- Testing various user flows (e.g., a customer submitting a new complaint, an agent updating a complaint's status, an admin assigning a complaint) helps confirm that all processes are functioning as intended.

Output Screenshots:

- **Landing Page:** A clean, professional landing page that welcomes users and provides clear "Login" and "Sign Up" buttons.



- **Registration/Login Pages:** Simple, secure forms for user authentication.

The screenshot shows a web browser window with two tabs: 'PROJECT REPORT ON.docx' and 'Online Complaints'. The address bar displays 'smartbridge-project-frontend.onrender.com/register'. The page features a light blue background with a white registration form in the center. The form is titled 'Create your account' with a link 'Already have an account? Sign in'. It includes input fields for 'Full Name' (containing 'GOUNDA'), 'Email', 'Password', and 'Confirm Password'. There is a dropdown menu for 'Account Type' with the text 'Select Account Type'. A blue 'Create Account' button is at the bottom of the form. The Windows taskbar at the bottom shows the date and time as 9:29 PM on 7/17/2025, along with various system icons and a weather widget indicating 'Extremely humid Now'.

Create your account
[Already have an account? Sign in](#)

Full Name

Email

Password

Confirm Password

Account Type

[Create Account](#)

The screenshot shows the same web browser window with the address bar displaying 'smartbridge-project-frontend.onrender.com/login'. The page features a light blue background with a white login form in the center. The form is titled 'Sign in to your account' with a link 'Or create a new account'. It includes input fields for 'Email' and 'Password'. A blue 'Sign In' button is at the bottom of the form. The Windows taskbar at the bottom shows the date and time as 9:29 PM on 7/17/2025, along with various system icons and a weather widget indicating 'Extremely humid Now'.

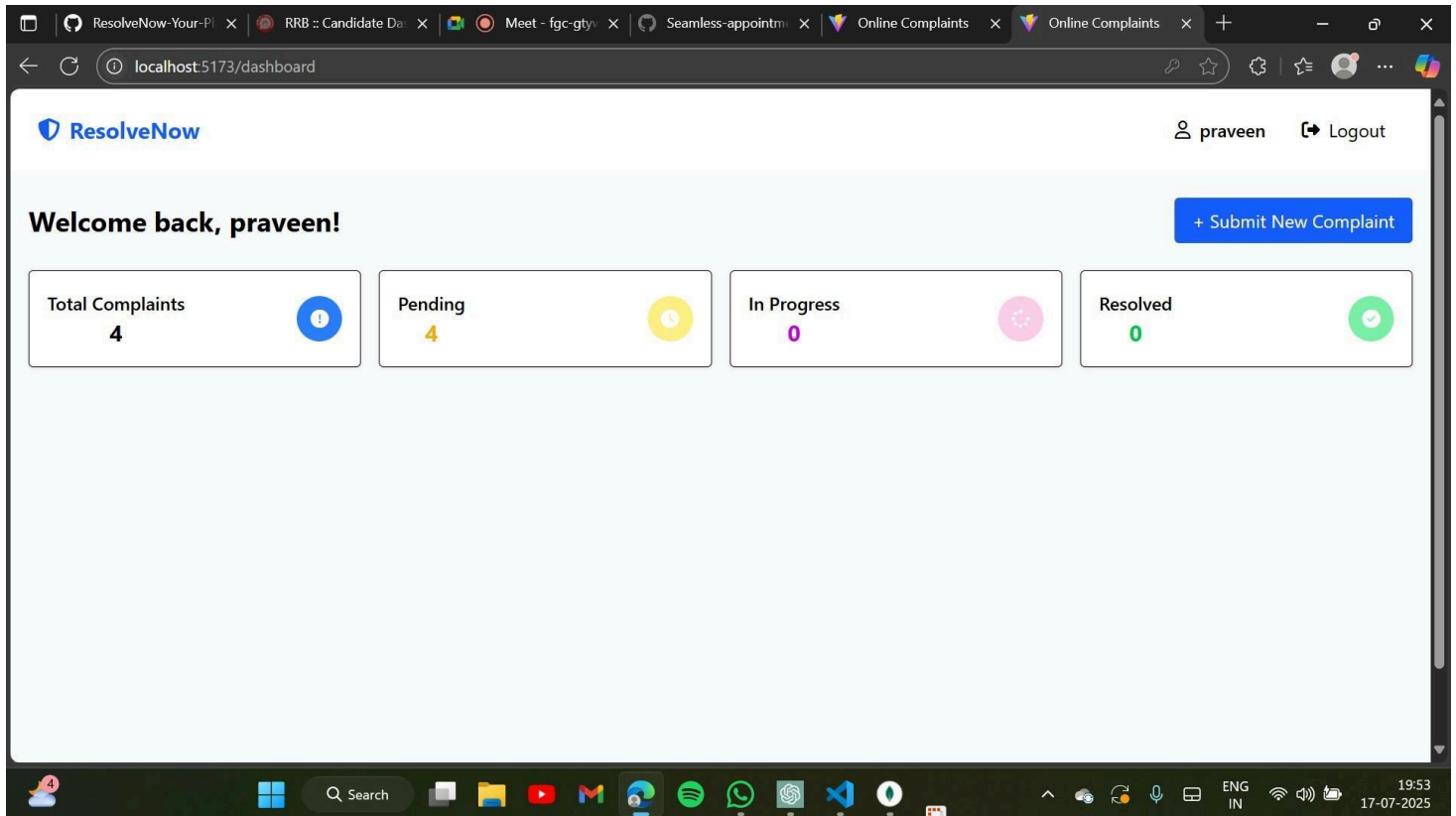
Sign in to your account
[Or create a new account](#)

Email

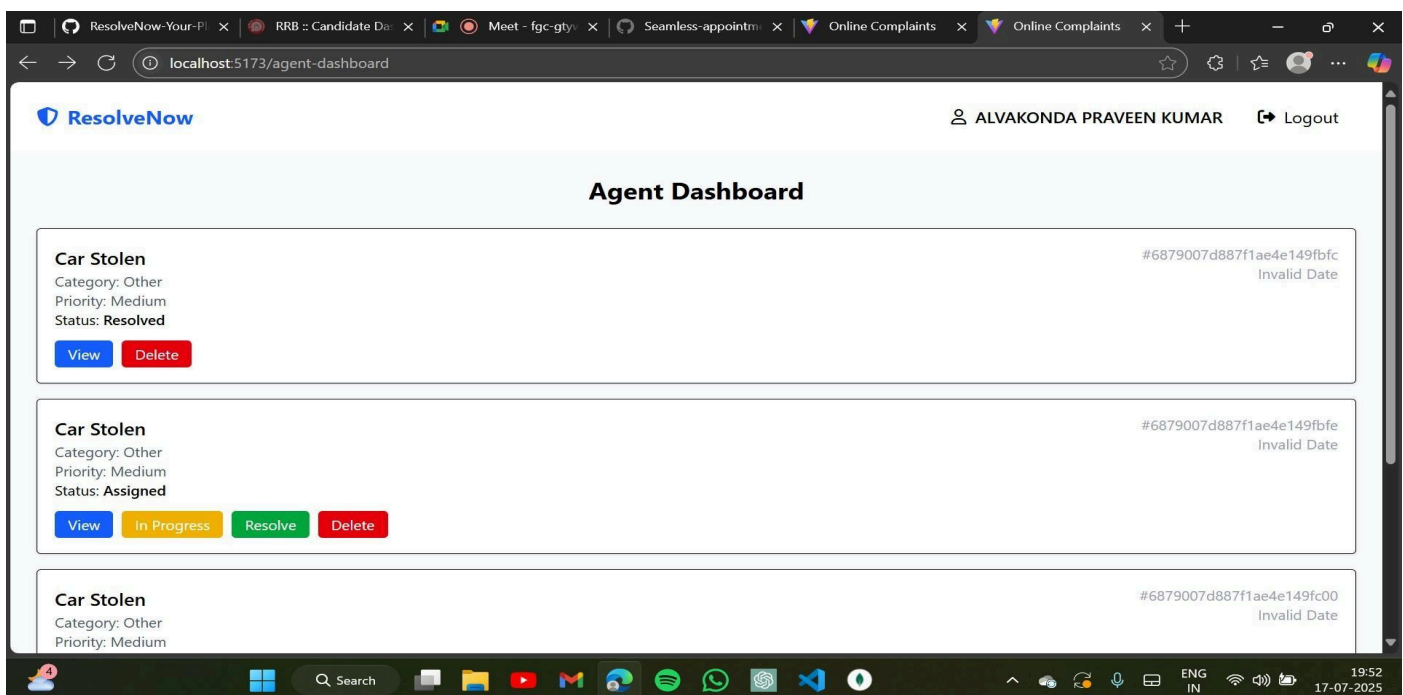
Password

[Sign In](#)

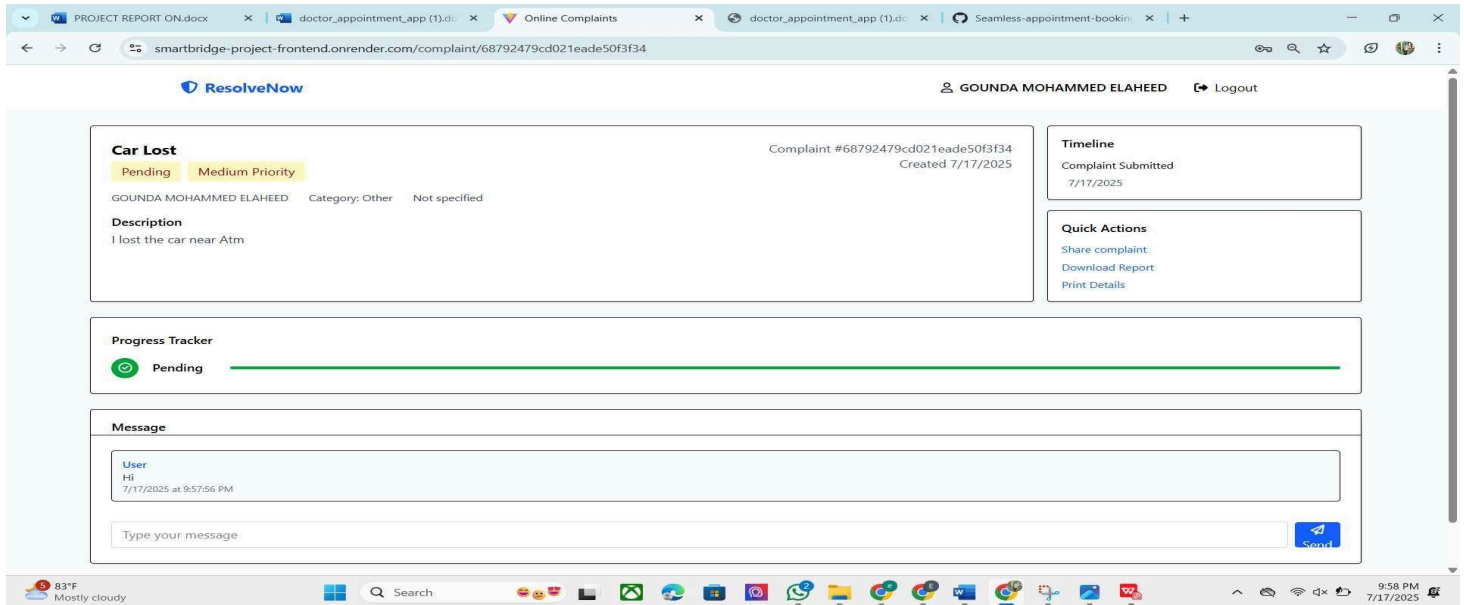
- **Customer Dashboard:** A personalized dashboard showing a list of submitted complaints with their current status. Includes a prominent button to "File a New Complaint."



- **Agent Dashboard:**



- **Complaint Detail View:** A comprehensive page showing all information for a single complaint, including the messaging interface for communication.



CONCLUSION

The **ResolveNow** project successfully achieves its objective of creating a modern, efficient, and user-friendly online complaint management system. By leveraging the MERN stack, the platform provides a robust and scalable solution that addresses the shortcomings of traditional complaint-handling methods. It centralizes communication, enhances transparency, and empowers organizations to resolve customer issues more effectively. This project serves as a strong example of applying modern web technologies to solve tangible business problems and improve the customer experience.